



ÉCOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE ET D'ANALYSE DES SYSTÈMES

- RABAT -

Filière : Cybersécurité, Cloud et Mobile Computing

SecureWallet : Application Mobile de Gestion Sécurisée de Cartes Bancaires

---

Réalisé par :
Amal BAKHOUYA
Saida EL MACHMACHI

Encadré par :
Pr. Wissal ATTAOUI

Année universitaire : 2025 – 2026

Remerciements

Avant tout, nous tenons à exprimer notre profonde gratitude à notre encadrante, **Mme Wissal ATTAOUI**, pour sa disponibilité, ses orientations précieuses et ses conseils judicieux tout au long de la réalisation de ce projet. Son expertise et son soutien technique nous ont guidés efficacement dans la mise en œuvre des briques de sécurité de notre application.

Nous adressons également nos vifs remerciements à l'ensemble du corps professoral de l'**École Nationale Supérieure d'Informatique et d'Analyse des Systèmes (ENSIAS)**, qui nous dispense une formation d'excellence et nous transmet les compétences indispensables pour aborder le monde de la cybersécurité avec rigueur et professionnalisme.

Enfin, nous remercions chaleureusement nos familles, nos proches et nos camarades de promotion pour leur soutien moral constant, leurs encouragements et leur aide précieuse durant cette période de travail.

Amal BAKHOUYA & Saida EL MACHMACHI

Résumé

Ce rapport présente la conception et le développement de **SecureWallet**, une application mobile Android dédiée à la gestion sécurisée de cartes bancaires[cite: 1]. Développé dans un cadre académique, ce projet met en œuvre une stratégie de défense en profondeur afin de pallier les vulnérabilités inhérentes aux terminaux mobiles[cite: 1, 2]. L'architecture logicielle s'appuie sur quatre piliers majeurs : une authentification centralisée via *Firebase Authentication*[cite: 1, 2], un chiffrement fort des données au repos par le biais du couple algorithmique *AES-GCM* dont la clé est isolée au sein de l'*AndroidKeyStore*[cite: 1, 2], une persistance locale sous *SQLite* partitionnée strictement par identifiant utilisateur (`UID`)[cite: 1, 2], et enfin un module de contrôle d'intégrité (*SecurityChecker*) capable de détecter les environnements compromis (Root, émulateurs ou débogage actif)[cite: 1, 2]. Les phases de test ont démontré la robustesse et la viabilité de l'approche applicative retenue, garantissant l'étanchéité des données face aux attaques courantes[cite: 2].

Mots-clés : Sécurité Mobile, Android, Cryptographie, AES-GCM, AndroidKeyStore, Firebase, SQLite, Intégrité du dispositif.

Abstract

This report describes the design and development of **SecureWallet**, a secure Android mobile application for managing bank cards[cite: 1]. Developed within an academic framework, this project implements a defense-in-depth strategy to mitigate vulnerabilities inherent to mobile devices[cite: 1, 2]. The software architecture is built upon four major pillars: centralized cloud authentication via *Firebase Authentication*[cite: 1, 2], robust data-at-rest encryption utilizing the *AES-GCM* cryptographic algorithm with keys hardware-isolated in the *AndroidKeyStore*[cite: 1, 2], a local *SQLite* storage strictly partitioned by user unique identifier (`UID`)[cite: 1, 2], and an anti-tampering integrity verification module (*SecurityChecker*) designed to detect compromised execution environments (Root access, emulators, or active debugging)[cite: 1, 2]. Qualification and validation tests demonstrated the robustness and compliance of our implementation, ensuring strict data containment and resilience against common mobile threat vectors[cite: 2].

Keywords: Mobile Security, Android, Cryptography, AES-GCM, AndroidKeyStore, Firebase, SQLite, Device Integrity.

Contents

Introduction Générale **1**

1 Introduction et Contexte du Projet **2**
1.1 Présentation générale 3
1.2 Objectifs pédagogiques 3
1.3 Choix du thème 4
1.4 Environnement de développement 4

2 Architecture et Conception du Système **5**
2.1 Vue d'ensemble de l'architecture 6
2.2 Flux de navigation sécurisé 6
2.3 Structure du Code Source 7
2.3.1 Arborescence globale du projet 7
2.3.2 Isolation des données par partitionnement UID 7

3 Authentification et Gestion des Utilisateurs **8**
3.1 Pourquoi Firebase Authentication ? 9
3.2 Configuration du projet 9
3.3 Implémentation de la Connexion (LoginActivity) 9
3.4 Implémentation de l'Inscription (RegisterActivity) 10
3.5 Gestion de la session utilisateur 10

4 Sécurité des Données (Chiffrement et Stockage) **11**
4.1 Principe du chiffrement des données 12
4.2 Choix algorithmiques : AES-GCM 12
4.3 Gestion de la clé via AndroidKeyStore 12
4.3.1 Génération et récupération de la clé 12
4.3.2 Opération de chiffrement et déchiffrement 13
4.4 Stockage Local Sécurisé (SQLite + Chiffrement applicatif) 14
4.4.1 Comparaison des approches 14
4.4.2 Schéma de la base de données (DatabaseHelper) 14
4.4.3 Flux d'écriture et de lecture 15

5 Intégrité du Dispositif et Fonctionnalités **16**

5.1 Vérification de l'intégrité (SecurityChecker) 17
5.1.1 Pourquoi vérifier l'intégrité ? 17
5.1.2 Vérifications implémentées 17
5.1.3 Exemple de détection du Root 17
5.1.4 Réaction et comportement de l'application 18
5.2 Fonctionnalités de l'Application (Description des Écrans) 18

6 Tests, Résultats et Validation **20**
6.1 Environnement de test 21
6.2 Scénarios de tests et résultats obtenus 22
6.3 Problèmes rencontrés et solutions apportées 23
6.4 Problèmes rencontrés et solutions apportées 26

Conclusion Générale **28**

Introduction Générale

Dans le paysage informatique moderne, la sécurisation des applications mobiles manipulant des données financières est un défi central. Notre projet vise à étudier et mettre en place une architecture logicielle hautement sécurisée pour un portefeuille numérique sous environnement Android.

En effet, nous sommes confrontés à une réalité où l'adoption croissante des transactions mobiles par les utilisateurs entraîne des préoccupations majeures en matière de cybercriminalité. La complexité croissante des attaques sur Android nécessite des stratégies de sécurité et de défense en profondeur efficaces pour faire face aux fuites de données potentielles et aux vulnérabilités des terminaux.

Notre objectif est de concevoir une application sécurisée nommée **SecureWallet** pour la gestion de cartes bancaires, en mettant l'accent sur la confidentialité, l'authenticité et l'intégrité des informations stockées. Nous cherchons à fournir une implémentation logicielle claire et des solutions pratiques pour garantir un niveau de protection optimal tout au long du cycle de vie des données bancaires locales.

Notre projet se structure en plusieurs volets. Dans un premier chapitre, nous situons le projet dans son contexte général en présentant sa problématique, ses objectifs pédagogiques et son périmètre technique. Ensuite, dans le chapitre 2, "Architecture et Conception du Système", nous examinons la structure globale en couches de l'application ainsi que son flux de navigation sécurisé. Le troisième chapitre se concentre sur la mise en œuvre pratique de la gestion des identités via les services de *Firebase Authentication*. Le quatrième chapitre détaille les mécanismes de chiffrement fort avec le couple *AES-GCM* et l'*AndroidKeyStore*, couplés à la persistance des données au sein d'une base *SQLite*. Le cinquième chapitre traite de la vérification de la légitimité du dispositif d'exécution à l'aide d'un module anti-tampering (*SecurityChecker*) ainsi que de la présentation fonctionnelle des écrans. Enfin, le dernier chapitre évalue l'efficacité de nos solutions à travers une matrice de scénarios de tests et documente les incidents techniques résolus au cours du développement.

Chapitre 1

Introduction et Contexte du Projet

1.1 Présentation générale

L'application **SecureWallet** est un portefeuille numérique mobile développé spécifiquement dans le cadre des travaux pratiques de sécurité des applications mobiles (TP5 et TP6)[cite: 17]. Face à la multiplication des cybermenaces ciblant l'écosystème Android, ce projet met en œuvre des mécanismes de défense en profondeur pour protéger des données hautement critiques : les coordonnées bancaires[cite: 18].

SecureWallet offre à un utilisateur préalablement authentifié une interface lui permettant de stocker, consulter et supprimer ses cartes bancaires en toute sérénité[cite: 18]. Pour ce faire, l'application combine plusieurs barrières de protection complémentaires, garantissant la confidentialité, l'intégrité et l'authenticité des informations à chaque étape de leur cycle de vie[cite: 18, 23].

1.2 Objectifs pédagogiques

La réalisation de cette application vise l'acquisition et la mise en pratique de compétences avancées en développement Android sécurisé. Les objectifs principaux s'articulent autour de cinq axes technologiques :

- **Gestion des identités :** Implémenter un système d'authentification robuste et distant (via courriel et mot de passe) en s'appuyant sur les services de *Firebase Authentication*[cite: 20].
- **Cryptographie embarquée :** Maîtriser le chiffrement des flux et des données sensibles à l'aide du couple algorithmique AES-GCM, combiné à la gestion matérielle ou logicielle des clés via l'API *Android KeyStore*[cite: 21].
- **Isolation des données :** Concevoir un espace de stockage local persistant sous *SQLite*, structuré de manière à isoler de façon hermétique les données propres à chaque utilisateur[cite: 22].
- **Contrôle d'intégrité (Anti-tampering) :** Développer un module de détection (*SecurityChecker*) capable d'évaluer la légitimité de l'environnement d'exécution au démarrage (détection du Root, d'émulateurs ou de descripteurs de débogage actifs)[cite: 23].
- **Bonnes pratiques architecturales :** Structurer un code source Java propre, lisible et hautement maintenable, conforme aux exigences de sécurité industrielles[cite: 24].

1.3 Choix du thème

Afin de valider les exigences de sécurité imposées par l'encadrement, quatre thématiques distinctes ont fait l'objet d'une évaluation comparative : gestionnaire de mots de passe, coffre-fort de documents, application médicale et portefeuille de cartes[cite: 26].

Le choix s'est arrêté sur le portefeuille de cartes bancaires **SecureWallet**[cite: 27]. Ce thème légitime de façon naturelle l'intégration stricte de l'intégralité des briques de sécurité requises[cite: 27]. En effet, manipuler des numéros de cartes de crédit, des codes CVV et des dates d'expiration impose obligatoirement un chiffrement fort, une isolation stricte par compte utilisateur pour éviter les fuites collatérales, ainsi qu'une vérification intransigeante de la santé globale du terminal mobile[cite: 27].

1.4 Environnement de développement

Le développement et la validation technique de l'application ont été menés au sein de l'écosystème résumé dans le tableau 1.1 ci-dessous[cite: 29]:

Élément	**Valeur et Justification réglementaire**
IDE	Android Studio (dernière version stable) [cite: 29]
Langage	Java (migration depuis Kotlin en cours de développement) [cite: 29]
Build System	Gradle (Groovy DSL) – `compileSdk` / `targetSdk` 36 [cite: 29]
Émulateur	AVD API 30 (Android 11) – Choix stratégique pour la stabilité des performances [cite: 29]
Backend Auth	Firebase Authentication (Console Firebase configurée en ligne) [cite: 29]
Fichier config	Fichier `google-services.json` injecté à la racine du module `/app/` [cite: 29]

Table 1.1: Environnement technique de développement SecureWallet

Chapitre 2

Architecture et Conception du Système

2.1 Vue d'ensemble de l'architecture

Pour répondre aux contraintes de complexité minimale imposées dans le cadre du projet, l'application **SecureWallet** adopte une architecture en couches simples et directes[cite: 37]. Afin de maximiser la lisibilité du code source et de maintenir un flux d'exécution fluide, aucun pattern architectural complexe (tel que MVVM ou MVP) n'a été volontairement appliqué : la logique métier essentielle reste intégrée au sein des composants *Activities*[cite: 38].

L'organisation logicielle est segmentée en quatre couches logiques interdépendantes décrites dans le tableau 2.1[cite: 39]:

Couche	Composants clés	Rôle et responsabilités
Présentation	`LoginActivity`, `RegisterActivity`, `MainActivity`, `AddCardActivity`, `CardDetailActivity`	Interfaces utilisateur, navigation entre écrans[cite: 39].
Métier / Sécurité	`CryptoManager`, `SecurityChecker`	Chiffrement AES-GCM, vérification d'intégrité[cite: 39].
Données	`DatabaseHelper`, `Card`, `CardAdapter`	Accès SQLite, modèle de carte, affichage RecyclerView[cite: 39].
Auth externe	*Firebase Authentication*	Gestion identité, UID utilisateur[cite: 39].

Table 2.1: Structure en couches de l'application SecureWallet

2.2 Flux de navigation sécurisé

La cinématique d'utilisation de l'application suit un flux strictement linéaire et validé par des points de contrôle de sécurité à chaque transition d'écran[cite: 41]:

1. **Lancement :** Dès l'initialisation de l'application, le module `SecurityChecker` s'exécute pour analyser l'appareil (vérification root, émulateur, débogage) et lève une alerte en cas d'environnement compromis[cite: 42].

2. **Identification :** L'utilisateur accède à la `LoginActivity` où ses identifiants sont validés par *Firebase*, menant en cas de succès à la `MainActivity`[cite: 43]. En cas de besoin, il bascule vers la `RegisterActivity` pour la création de compte[cite: 44].

3. **Écran Principal :** Après authentification réussie, la `MainActivity` affiche la liste des cartes filtrée via une `RecyclerView` et offre un bouton d'ajout[cite: 45].

4. **Ajout de carte :** Via un bouton d'action, l'utilisateur ouvre `AddCardActivity`[cite: 45]. Lors de la validation, les données subissent un chiffrement via `CryptoManager`

avant l'insertion au sein de la base SQLite[cite: 46, 47, 48].

5. **Consultation :** Un clic sur un élément de la liste redirige vers la `CardDetailActivity` qui opère le déchiffrement pour l'affichage et permet la suppression[cite: 49, 50].

2.3 Structure du Code Source

2.3.1 Arborescence globale du projet

Afin de refléter l'organisation modulaire du projet, l'arborescence des classes Java sous le package racine `com.example.securewallet` se présente comme suit[cite: 195, 197]:

```
app/src/main/java/com/example/securewallet/
 model/
    Card.java                  # Modèle (POJO) de l'entité carte
 adapter/
    CardAdapter.java           # RecyclerView Adapter
 database/
    DatabaseHelper.java        # SQLiteOpenHelper natif
 security/
    CryptoManager.java         # Moteur de chiffrement KeyStore
    SecurityChecker.java       # Service Sécurité (Root/Ému)
 ui/
     LoginActivity.java         # Activity Écran de connexion
     RegisterActivity.java      # Activity Écran d'inscription
     MainActivity.java          # Activity Écran principal
     AddCardActivity.java       # Activity Formulaire d'ajout
     CardDetailActivity.java    # Activity Affichage détail
```

Figure 2.1: Arborescence logique des classes du projet SecureWallet

2.3.2 Isolation des données par partitionnement UID

L'une des exigences clés d'étanchéité logicielle repose sur le fait qu'un utilisateur ne doit sous aucun prétexte accéder aux données d'un tiers partageant le même appareil[cite: 53].

Pour implémenter cette isolation, le schéma de la base de données intègre un attribut de filtrage strict nommé $\texttt{user}_id, qui accueille la valeur de l'UID unique fourni par Firebase Authentication$ 52, 53]. *Toutes les requtes de selection filtrent strictement sur le compte actif [cite : 53] :*

```sql
SELECT * FROM cards WHERE user_id = ?; -- L'UID Firebase passé en paramètre
```

CHAPITRE 3

Authentification et Gestion des Utilisateurs

3.1 Pourquoi Firebase Authentication ?

La sécurisation d'une application de type portefeuille numérique impose d'identifier formellement chaque utilisateur avant de lui accorder l'accès aux données.
Pour éviter le stockage local ou le développement risqué d'un backend d'authentificati personnalisé, le choix s'est porté sur *Firebase Authentication*[cite: 61].

Cette solution cloud offre plusieurs garanties de sécurité indispensables :

- Transmission sécurisée et gestion complète : Elle gère entièrement le cycle de vie des identités (inscription, connexion, réinitialisation)[cite: 61].
- Identifiant unique (UID) : Elle génère un UID unique et persistant par compte, utilisé comme clé d'isolation des données au sein de notre base SQLite locale[cite: 62].

3.2 Configuration du projet

L'intégration de Firebase nécessite l'enchaînement de plusieurs configurations techniques[cite: 64, 65, 66, 67, 68]:

- Création du projet sur la console Firebase et activation de la méthode Email/Mot de passe[cite: 64, 65].
- Placement du fichier google-services.json dans le répertoire /app/[cite: 66].
- Ajout du plugin Google Services et de la dépendance correspondante[cite: 67, 68].

```
dependencies {
    // Dépendance pour Firebase Authentication
    implementation 'com.google.firebase:firebase-auth:22.x'
}
```

3.3 Implémentation de la Connexion (LoginActivity)

La classe LoginActivity gère la connexion[cite: 70]. Le code source s'appuie sur l'instance FirebaseAuth pour soumettre les données via signInWithEmailAndPassword 71]:

```
FirebaseAuth mAuth = FirebaseAuth.getInstance();
mAuth.signInWithEmailAndPassword(email, password)
    .addOnCompleteListener(task -> {
        if (task.isSuccessful()) {
            // Récupérer l'UID et lancer MainActivity
            String uid = mAuth.getCurrentUser().getUid();
            startActivity(new Intent(this, MainActivity.class));
        } else {
            Toast.makeText(this, "Échec de connexion", LENGTH_SHORT).show();
        }
    });
```

3.4 Implémentation de l'Inscription (RegisterActivity)

La classe RegisterActivity utilise la méthode createUserWithEmailAndPassword[cite: 73]. En cas de succès, l'utilisateur est redirigé vers la LoginActivity pour se connecter explicitement, ceci pour des raisons de clarté pédagogique[cite: 73].

3.5 Gestion de la session utilisateur

Firebase maintient la session automatiquement entre les relances de l'application[cite: 75]. Dans la MainActivity, on vérifie si un utilisateur est connecté au lancement[cite: 76]:

```
if (FirebaseAuth.getInstance().getCurrentUser() == null) {
    startActivity(new Intent(this, LoginActivity.class));
    finish();
}
```

Le bouton de déconnexion appelle la méthode FirebaseAuth.getInstance().signOut() puis redirige l'utilisateur vers la LoginActivity[cite: 78].

CHAPITRE 4

Sécurité des Données (Chiffrement et Stockage)

4.1 Principe du chiffrement des données

Avant toute insertion au sein de la base de données locale, les informations hautement sensibles de chaque carte bancaire (à savoir le numéro de carte, le code CVV et la date d'expiration) subissent une opération de chiffrement[cite: 86].

La clé de chiffrement est stockée exclusivement dans l'*AndroidKeyStore*, un conteneur matériel ou logiciel isolé de façon hermétique du processus applicatif standard[cite: 87]. Cela garantit que la clé ne quitte jamais le keystore : seules les opérations de traitement cryptographique y sont exécutées, barrant la route aux attaques par analyse de la mémoire vive[cite: 87].

4.2 Choix algorithmiques : AES-GCM

Le moteur de sécurité de SecureWallet s'appuie sur l'algorithme AES (Advanced Encryption Standard) configuré en mode **GCM** (Galois/Counter Mode) avec une longueur de clé de 256 bits[cite: 89, 90]. Ce choix est motivé par plusieurs propriétés fondamentales :

- Confidentialité et Authenticité : Contrairement aux modes plus anciens (comme CBC), le mode GCM fournit un chiffrement authentifié (AEAD). Il génère un tag d'authentification (MAC) permettant de détecter immédiatement toute tentative d'altération malveillante des données stockées[cite: 89, 91].
- Vecteur d'initialisation (IV) unique : Chaque opération de chiffrement génère et utilise un IV aléatoire de 12 octets[cite: 93]. Cela assure que deux cartes possédant le même numéro de structure produiront des ciphertexts totalement distincts en base de données[cite: 93].
- Conformité aux standards : Le couple AES-GCM représente la recommandation officielle du NIST et de Google pour la sécurisation des architectures mobiles modernes[cite: 92].

4.3 Gestion de la clé via AndroidKeyStore

4.3.1 Génération et récupération de la clé

La clé AES est générée une unique fois lors du tout premier démarrage de l'application sous l'alias "SecureWalletKey"[cite: 99]. Les lancements ultérieurs récupèrent directement cette instance matérielle[cite: 99]. L'initialisation s'exécute ainsi au sein de la classe CryptoManager[cite: 94, 100]:

```
KeyStore keyStore = KeyStore.getInstance("AndroidKeyStore");
keyStore.load(null);

if (!keyStore.containsAlias("SecureWalletKey")) {
    KeyGenerator kg = KeyGenerator.getInstance(
        KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore");
    kg.init(new KeyGenParameterSpec.Builder("SecureWalletKey",
            KeyProperties.PURPOSE_ENCRYPT | KeyProperties.PURPOSE_DECRYPT)
        .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
        .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
        .setKeySize(256)
        .build());
    kg.generateKey();
}
```

4.3.2 Opération de chiffrement et déchiffrement

Lors de l'encapsulation, l'IV de 12 octets généré dynamiquement est extrait puis concaténé en tête du bloc chiffré (*ciphertext*) avant d'encoder le tout en chaîne Base64 pour le stockage en base[cite: 105, 111]:

```
// Extrait de la méthode encrypt de CryptoManager
Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
cipher.init(Cipher.ENCRYPT_MODE, secretKey);
byte[] iv = cipher.getIV(); // 12 octets
byte[] ciphertext = cipher.doFinal(plaintext.getBytes("UTF-8"));

byte[] combined = new byte[iv.length + ciphertext.length];
System.arraycopy(iv, 0, combined, 0, iv.length);
System.arraycopy(ciphertext, 0, combined, iv.length, ciphertext.length);
return Base64.encodeToString(combined, Base64.DEFAULT);
```

Pour l'opération inverse, la méthode de déchiffrement isole d'abord les 12 premiers octets pour reconstruire la spécification des paramètres GCM avant de traiter le reste du flux[cite: 111, 112]:

```
// Extrait de la méthode decrypt de CryptoManager
byte[] combined = Base64.decode(encryptedData, Base64.DEFAULT);
byte[] iv = Arrays.copyOfRange(combined, 0, 12);
byte[] ciphertext = Arrays.copyOfRange(combined, 12, combined.length);
```

```
GCMParameterSpec spec = new GCMParameterSpec(128, iv);
cipher.init(Cipher.DECRYPT_MODE, secretKey, spec);
return new String(cipher.doFinal(ciphertext), "UTF-8");
```

4.4 Stockage Local Sécurisé (SQLite + Chiffrement applicatif)

4.4.1 Comparaison des approches

Plutôt que d'intégrer une lourde bibliothèque externe comme SQLCipher, le projet implémente un stockage persistant basé sur le moteur SQLite natif d'Android couplé au chiffrement chirurgical des attributs par le CryptoManager[cite: 120]. Ce choix se justifie par l'analyse comparative suivante[cite: 123]:

Critère	**SQLCipher**	**SQLite + CryptoManager (Retenu)**
Périmètre	Chiffrement de la base entière	Champs sensibles individuels [cite: 123]
Complexité	Dépendance tierce + configuration NDK	Code Java pur natif, zéro dépendance [cite: 123]
Performance	Surcharge globale liée au déchiffrement complet	Surcharge minimale (un champ à la fois) [cite: 123]
Code	Lisibilité moyenne	Excellente et modulaire [cite: 123]

Table 4.1: Comparaison des stratégies de stockage local sécurisé

4.4.2 Schéma de la base de données (DatabaseHelper)

La classe DatabaseHelper hérite de SQLiteOpenHelper[cite: 125]. Elle orchestre la création de la table cards dont la structure applicative stocke de pures chaînes Base64 chiffrées pour les colonnes sensibles[cite: 126]:

```
CREATE TABLE cards (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id TEXT NOT NULL,       -- UID Firebase pour l'isolation
    card_name TEXT NOT NULL,     -- En clair (ex: 'Visa Pro')
    card_number TEXT NOT NULL,   -- Chiffré AES-GCM
    expiry_date TEXT NOT NULL,   -- Chiffré AES-GCM
    cvv TEXT NOT NULL            -- Chiffré AES-GCM
);
```

4.4.3 Flux d'écriture et de lecture

La manipulation des données respecte une étanchéité stricte entre les couches de l'application :

- Flux d'écriture (AddCardActivity) : Les entrées utilisateur saisies au clavier sont immédiatement interceptées[cite: 128]. Chaque champ sensible est chiffré via l'appel à cryptoManager.encrypt()[cite: 130]. Un modèle Card associant ces chaînes masquées et l'UID de la session en cours est assemblé puis injecté en base via dbHelper.addCard(card)[cite: 131, 132].
- Flux de lecture (CardDetailActivity) : Lors du clic sur une entité, la ligne est requêtée depuis la base par son identifiant unique[cite: 135]. L'application invoque cryptoManager.decrypt() sur les données brutes, puis injecte le résultat déchiffré à la volée dans les composants graphiques (TextView) pour l'affichage final[cite: 136, 137].

CHAPITRE 5

Intégrité du Dispositif et Fonctionnalités

5.1 Vérification de l'intégrité (SecurityChecker)

5.1.1 Pourquoi vérifier l'intégrité ?

Une application manipulant des informations bancaires critiques ne doit sous aucun prétexte s'exécuter sur un système d'exploitation dont les barrières de protection natives ont été altérées[cite: 2]. Un terminal mobile rooté, l'exécution au sein d'un émulateur non autorisé ou la présence d'un débogage actif constituent des vecteurs d'attaque majeurs[cite: 2]. Ces configurations permissives facilitent grandement l'extraction de clés cryptographiques, le détournement d'API (*hooking*) ou l'analyse dynamique de la mémoire vive applicative[ci 2].

Pour contrer ces menaces, la classe SecurityChecker réalise un audit complet des indicateurs de compromission dès le démarrage de l'application[cite: 2].

5.1.2 Vérifications implémentées

Le module de contrôle examine de manière séquentielle trois types de risques distincts[cite: 2] :

Vérification	Méthode correspondante	Risques et indicateurs détectés
Détection du Root	`isRooted()`	Présence de binaires `su`, d'applications de super-utilisateur connues ou de répertoires système suspects[cite: 2].
Détection d'émulateur	`isEmulator()`	Analyse des propriétés matérielles (`Build.FINGERPRINT`, `Build.PRODUCT`, `Build.MODEL`) contenant le mot-clé `'emulator'`[cite: 2].
Mode débogage	`isDebuggable()`	Vérification de l'état de l'indicateur `ApplicationInfo.FLAG_DEBUGGABLE` au sein du manifeste[cite: 2].

Table 5.1: Matrice des contrôles d'intégrité du SecurityChecker

5.1.3 Exemple de détection du Root

La méthode isRooted() inspecte les chemins d'accès système les plus fréquemment modifiés lors du déploiement d'un privilège *SuperUser*[cite: 2] :

```
private boolean isRooted() {
    String[] paths = {
        "/system/app/Superuser.apk",
```

```
        "/sbin/su",
        "/system/bin/su",
        "/system/xbin/su",
        "/data/local/xbin/su"
    };
    for (String path : paths) {
        if (new File(path).exists()) return true;
    }
    return false;
}
```

5.1.4 Réaction et comportement de l'application

Lorsqu'une anomalie environnementale est détectée par le SecurityChecker, l'applicatio intercepte le cycle de vie habituel et affiche immédiatement une boîte de dialogue d'alerte (AlertDialog) informant l'utilisateur des risques encourus[cite: 2]. [backgroundcolor=bg, linecolor=maincolor, linewidth=1pt] Note pédagogique : Dans le cadre actuel de ce projet académique, l'utilisateur a la possibilité de passer outre l'avertissement pour permettre la continuité des évaluations sur machine de développement (les émulateurs de test de l'environnement AVD API 30 retournant systématiquement la valeur true à la détection d'émulateur)[cite: 2]. En environnement de production industrielle, ce comportement devrait être remplacé par un blocage strict et immédiat du lancement applicatif[cite: 2].

5.2 Fonctionnalités de l'Application (Description des Écrans)

L'expérience utilisateur au sein de SecureWallet est structurée autour de cinq interfaces fonctionnelles hautement spécialisées[cite: 2] :

- LoginActivity (Connexion) : Fournit une interface épurée comprenant des champs de saisie pour l'adresse e-mail et le mot de passe, adossés à des règles de validation de surface[cite: 2]. Elle orchestre l'appel asynchrone à la méthode Firebase signInWithEmailAndPassword et gère explicitement le traitement des erreurs réseau ou d'identifiants incorrects[cite: 2].
- RegisterActivity (Inscription) : Permet la création de nouveaux profils à l'aide des champs e-mail, mot de passe et confirmation[cite: 2]. Une vérification locale de la concordance des chaînes de caractères est effectuée côté client avant de soumettre la création au serveur distant[cite: 2].

- MainActivity (Tableau de bord principal) : Écran pivot de l'application affichant une RecyclerView alimentée par le composant CardAdapter[cite: 2]. Pour préserver la confidentialité visuelle, chaque carte est affichée uniquement via son nom et ses 4 derniers chiffres masqués[cite: 2]. Un bouton d'action flottant (*FAB*) permet d'initier un ajout, tandis qu'un bouton dédié gère la déconnexion[cite: 2].

- AddCardActivity (Ajout de Titre) : Formulaire strict de capture de données comprenant le nom du titulaire, le numéro de carte (16 chiffres), la date d'expiration (MM/AA) et le code de sécurité CVV[cite: 2]. Les validations contrôlent la longueur et le format des chaînes avant que le CryptoManager ne chiffre individuellement chaque entrée sensible[cite: 2].

- CardDetailActivity (Consultation et Révocation) : Cet écran extrait les données chiffrées de la base SQLite, sollicite le Keystore pour restituer les informations en clair et applique un formatage lisible avec espaces (XXXX XXXX XXXX XXXX)[cite: 2]. Il intègre également un bouton de suppression définitive sécurisé par une demande de confirmation graphique (AlertDialog)[cite: 2].

CHAPITRE 6

Tests, Résultats et Validation

6.1 Environnement de test

Afin de valider la conformité fonctionnelle et les exigences de sécurité de l'application SecureWallet, l'intégralité de la campagne de qualification technique a été menée au sein de l'environnement contrôlé suivant :

- Terminal de test : Émulateur Android Virtual Device (AVD).
- Version du système : API 30 (Android 11).

[backgroundcolor=bg, linecolor=maincolor, linewidth=1pt] Note d'ingénierie : Le choix technique de l'API 30 résulte d'une décision d'optimisation suite à de lourds ralentissements et des problèmes de performances constatés sur les versions d'APIs supérieures (API 31 et plus) lors de l'exécution des émulateurs sur la machine de développement. La version Android 11 offre le compromis idéal entre la légèreté d'exécution et le support complet des API de sécurité requises (*AndroidKeyStore* et *GCMParameterSpec*).

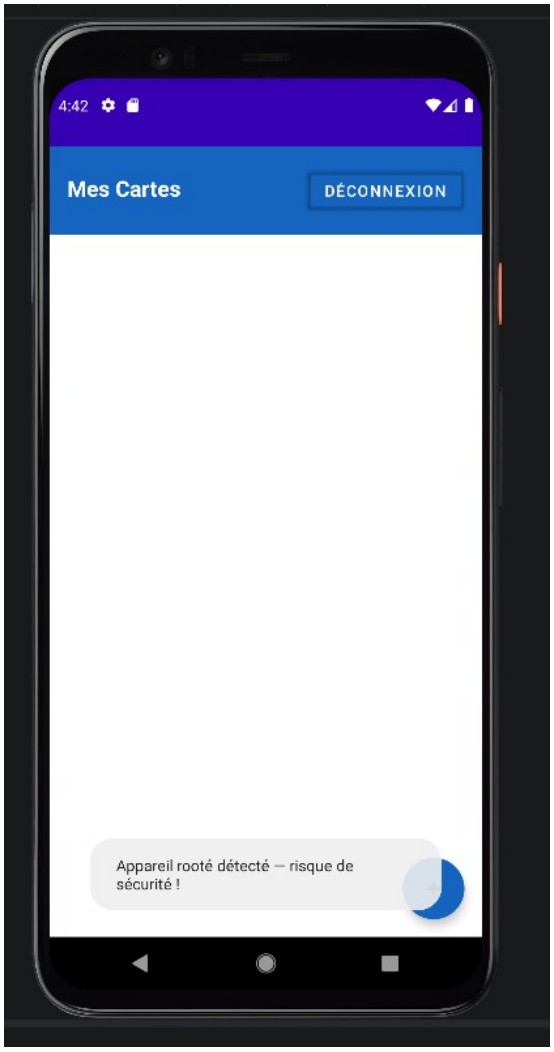


Figure 6.1: Notification Toast du SecurityChecker alertant de la détection d'un appareil rooté

6.2 Scénarios de tests et résultats obtenus

La validation du système s'est appuyée sur une matrice de tests couvrant le cycle de vie nominal de l'utilisateur, l'intégrité de la persistance des données et la robustesse des mécanismes de protection défensive.



Figure 6.2: Écran d'authentification SecureWallet

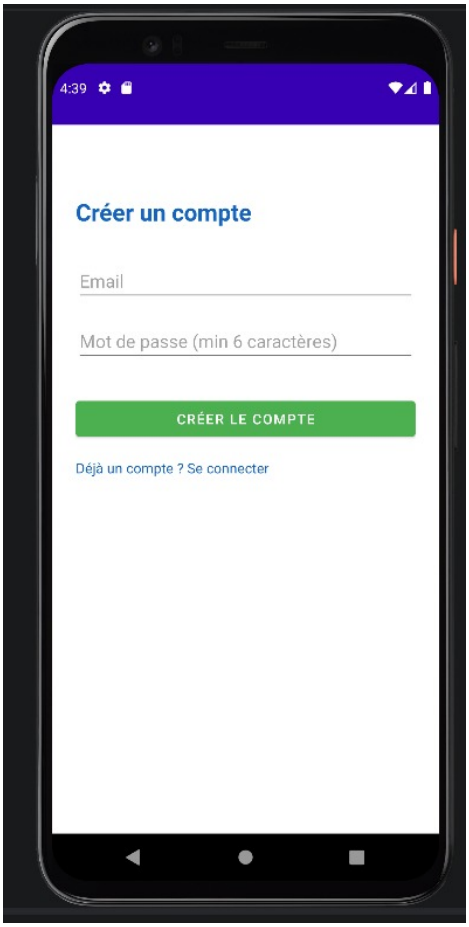


Figure 6.3: Interface de création de compte

Les résultats détaillés sont consignés dans le tableau 6.3 ci-dessous :

Scénario testé	Étapes de manipulation	Résultat attendu	Statut
Inscription	Saisie d'un e-mail valide et d'un mot de passe ≥ 6 caractères.	Compte enregistré sur Firebase, redirection vers `LoginActivity`.	**OK**
Inscription (Échec)	Saisie d'un e-mail malformé ou d'un mot de passe < 6 caractères.	Interception et affichage d'un message d'erreur Firebase.	**OK**
Connexion	Saisie du couple e-mail / mot de passe corrects.	Authentification Firebase réussie et accès à la `MainActivity`.	**OK**
Connexion (Échec)	Saisie d'un mauvais mot de passe ou tentative sur un compte inexistant.	Affichage d'un `Toast` d'erreur explicite.	**OK**
Ajout de carte	Remplissage complet du formulaire et clic sur sauvegarder.	Carte ajoutée en base et mise à jour de la `RecyclerView`.	**OK**
Chiffrement local	Extraction et analyse brute de la base de données SQLite via *adb*.	Informations sensibles illisibles en clair (chaînes Base64 chiffrées).	**OK**
Isolation multi-user	Connexion alternée avec deux comptes distincts sur le même appareil.	Chaque utilisateur ne visualise strictement que ses propres cartes.	**OK**
Détail et révocation	Clic sur une carte, puis demande de suppression avec confirmation.	Données déchiffrées affichées à la volée, puis retrait de la liste.	**OK**
SecurityChecker	Lancement de l'application sur l'émulateur AVD de test.	Détection de l'émulateur et déclenchement de l'alerte graphique.	**OK**
Session persistante	Relance de l'application sans action préable de déconnexion.	Bypass de l'écran de login et redirection directe vers l'accueil.	**OK**
Déconnexion	Clic sur le bouton de déconnexion de l'écran principal.	Destruction du jeton de session et retour à la `LoginActivity`.	**OK**

Table 6.1: Matrice des scénarios de validation et résultats de conformité

6.3 Problèmes rencontrés et solutions apportées

Le cycle de développement a été jalonné par plusieurs verrous techniques, principaleme
liés à des conflits de versions d'outils, des erreurs d'arborescence et des
exceptions d'exécution. Le tableau 6.4 synthétise les anomalies majeures rencontrées
ainsi que les correctifs appliqués :

Anomalie constatée	**Cause racine identifiée**	**Solution et correctif appliqués**
Gradle sync error	Valeur de `compileSdk` trop basse pour supporter les nouvelles dépendances de Firebase.	Élévation et alignement des paramètres `compileSdk` et `targetSdk` à la version 36.
Erreur de package au build	Classes Java positionnées incorrectement par rapport aux directives de package Android.	Réorganisation manuelle et déplacement rigoureux des fichiers dans la bonne arborescence de packages.
Attributs XML dupliqués	Génération automatique de portions de code redondantes (ex: déclarations multiples de dimensions).	Nettoyage et suppression manuelle des doublons au sein des fichiers de ressources `.xml` concernés.
Lenteur extrême de l'émulateur	Ressources matérielles système insuffisantes pour faire tourner les API récentes (API 33+).	Migration stratégique vers l'image AVD de l'API 30, beaucoup plus légère et stable.
NullPointerException sur Firebase	Invocation de la méthode `getCurrentUser()` avant la fin de l'initialisation de l'instance Firebase.	Restructuration du flux d'exécution pour garantir une instanciation non nulle avant tout accès aux données utilisateur.

Table 6.2: Registre de résolution des incidents techniques de développement

Scénario testé	Étapes de manipulation	Résultat attendu	Statut
Inscription	Saisie d'un e-mail valide et d'un mot de passe ≥ 6 caractères.	Compte enregistré sur Firebase, redirection vers `LoginActivity`.	**OK**
Inscription (Échec)	Saisie d'un e-mail malformé ou d'un mot de passe < 6 caractères.	Interception et affichage d'un message d'erreur Firebase.	**OK**
Connexion	Saisie du couple e-mail / mot de passe corrects.	Authentification Firebase réussie et accès à la `MainActivity`.	**OK**
Connexion (Échec)	Saisie d'un mauvais mot de passe ou tentative sur un compte inexistant.	Affichage d'un `Toast` d'erreur explicite.	**OK**
Ajout de carte	Remplissage complet du formulaire et clic sur sauvegarder.	Carte ajoutée en base et mise à jour de la `RecyclerView`.	**OK**
Chiffrement local	Extraction et analyse brute de la base de données SQLite via *adb*.	Informations sensibles illisibles en clair (chaînes Base64 chiffrées).	**OK**
Isolation multi-user	Connexion alternée avec deux comptes distincts sur le même appareil.	Chaque utilisateur ne visualise strictement que ses propres cartes.	**OK**
Détail et révocation	Clic sur une carte, puis demande de suppression avec confirmation.	Données déchiffrées affichées à la volée, puis retrait de la liste.	**OK**
SecurityChecker	Lancement de l'application sur l'émulateur AVD de test.	Détection de l'émulateur et déclenchement de l'alerte graphique.	**OK**
Session persistante	Relance de l'application sans action préalable de déconnexion.	Bypass de l'écran de login et redirection directe vers l'accueil.	**OK**
Déconnexion	Clic sur le bouton de déconnexion (*logout*) de l'écran principal.	Destruction du jeton de session et retour à la `LoginActivity`.	**OK**

Table 6.3: Matrice des scénarios de validation et résultats de conformité

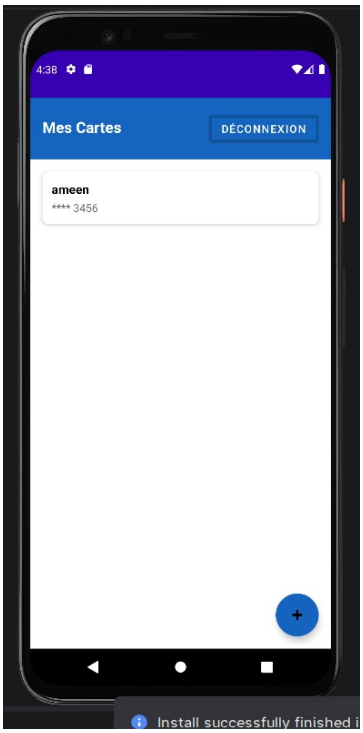


Figure 6.4: Liste principale avec masquage partiel

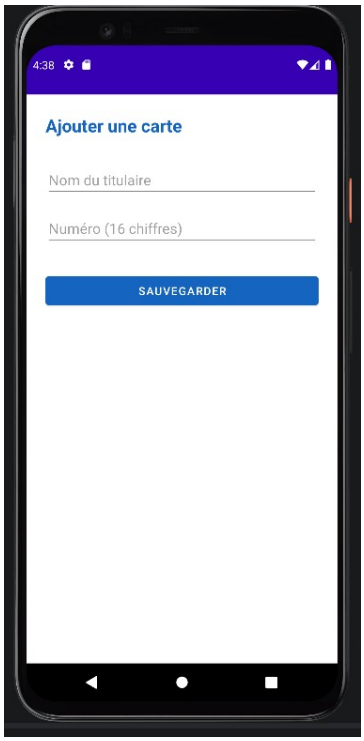


Figure 6.5: Formulaire d'ajout de carte



Figure 6.6: Consultation détaillée de la carte

6.4 Problèmes rencontrés et solutions apportées

Le cycle de développement a été jalonné par plusieurs verrous techniques, principaleme liés à des conflits de versions d'outils, des erreurs d'arborescence et des exceptions d'exécution [?]. Le tableau 6.4 synthétise les anomalies majeures rencontrées ainsi que les correctifs appliqués [?] :

Anomalie constatée	**Cause racine identifiée**	**Solution et correctif appliqués**
Gradle sync error	Valeur de `compileSdk` trop basse pour supporter les nouvelles dépendances de Firebase [?].	Élévation et alignement des paramètres `compileSdk` et `targetSdk` à la version 36 [?].
Erreur de package au build	Classes Java positionnées incorrectement par rapport aux directives de package Android [?].	Réorganisation manuelle et déplacement rigoureux des fichiers dans la bonne arborescence de packages [?].
Attributs XML dupliqués	Génération automatique de portions de code redondantes (ex: doubles déclarations de `layout_width/height`) [?].	Nettoyage et suppression manuelle des doublons au sein des fichiers de ressources `.xml` concernés [?].
Lenteur extrême de l'émulateur	Ressources matérielles système insuffisantes pour faire tourner les API récentes (API 33+) [?].	Migration stratégique vers l'image AVD de l'API 30, beaucoup plus légère et stable [?].
NullPointerException sur Firebase	Invocation de la méthode `getCurrentUser()` avant la fin de l'initialisation de l'instance Firebase [?].	Restructuration du flux d'exécution pour garantir une instanciation non nulle avant tout accès aux données utilisateur [?].

Table 6.4: Registre de résolution des incidents techniques de développement

Conclusion Générale

En somme, le développement de l'application SecureWallet a permis de répondre avec succès à l'ensemble des exigences de sécurité et de robustesse technique imposées dans le cadre des travaux pratiques de sécurité mobile (TP5/TP6)[cite: 220]. À travers la mise en place d'une stratégie de défense en profondeur, chaque brique technologique a été validée avec succès, qu'il s'agisse de la gestion distante des identités via *Firebase Authentication* [cite: 222], du partitionnement hermétique de la base *SQLite* par l'usage des identifiants UID uniques [cite: 222], de l'audit environnemental mené par le module SecurityChecker [cite: 222], ou encore du chiffrement fort des attributs sensibles par le couple algorithmique *AES-GCM* dont la clé maîtresse est idéalement isolée au sein de l'*AndroidKeyStore*[cite: 222].

Sur le plan pédagogique, ce projet académique a constitué un excellent levier d'apprentissage, favorisant l'acquisition de compétences solides en ingénierie Android. Il nous a permis d'assimiler des concepts fondamentaux allant de la manipulation de primitives cryptographiques de niveau industriel à la résolution de conflits d'intégration complexes liés à l'écosystème Gradle ou aux environnements d'émulation AVD[cite: 213, 230].

Bien que l'architecture actuelle réponde pleinement au cahier des charges initial [cite: 220], des perspectives d'amélioration restent envisageables pour élever l'application aux standards de production. Parmi ces pistes, nous retiendrons notamment l'intégration d'une couche de chiffrement globale via *SQLCipher* [cite: 224], le déploiement de l'authentification biométrique native [cite: 225], un comportement de blocage plus strict sur terminal compromis [cite: 227], ou encore la mise en œuvre de tests unitaires automatisés sous *JUnit* et *Mockito* pour pérenniser l'intégrité du code source[cite: 229].